

Especificação do Projeto Prático 01 (PP01)

Predição de Evasão Estudantil no Ensino Superior

Disciplina:	RUS0310 - Aprendizado de Máquina
Equipe:	04 (quatro) alunos no máximo
Entrega e apresentação:	30/09/2026 (Código + Relatório + Slides)
Linguagem:	Python 3 (recomendado)
Base de referência:	<i>Predict Students' Dropout and Academic Success</i> (UCI 697)

1 Apresentação e Objetivo

O objetivo deste projeto é integrar, em um único estudo experimental, os quatro primeiros assuntos da disciplina. O projeto parte de um **problema e de uma base de dados já fixados**. A equipe não escolhe o *dataset*: recebe o problema formulado e deve construir, por conta própria, um *pipeline* completo do dado bruto à recomendação final.

O problema

Uma instituição de ensino superior deseja **identificar precocemente estudantes em risco de evasão**, de modo a acionar tutoria, apoio financeiro ou acompanhamento pedagógico antes que a desistência se concretize.

A equipe deve construir um classificador que, a partir de dados sociodemográficos, socioeconômicos, de ingresso e de desempenho acadêmico, prediga o desfecho do estudante entre três categorias: *Dropout* (evadido), *Enrolled* (ainda matriculado, sem desfecho) e *Graduate* (concluente).

O ponto central do projeto **não é maximizar a acurácia**. É responder, com evidência experimental, a três perguntas: (i) *com que antecendência* é possível prever a evasão sem trapacear com informação do futuro; (ii) *quais características* realmente carregam sinal; e (iii) *qual modelo* entregar à coordenação, considerando desempenho, interpretabilidade e custo.

2 Regra de Originalidade

Leia antes de começar

Esta base é extremamente popular no Kaggle e no GitHub. Existem centenas de *notebooks* públicos resolvendo exatamente este problema. O que se avalia aqui não é o resultado, que é conhecido, mas o raciocínio experimental da equipe e a implementação própria dos algoritmos.

O que significa “mesmo problema, trabalho autoral”:

- **Deve ser igual:** a base de dados, os três cenários de predição (Seção 3.1), os dois algoritmos obrigatórios (árvore de decisão e *k*-NN), o protocolo experimental (Seção 4.5) e os experimentos da Seção 6.
- **Deve ser diferente:** a engenharia de atributos, os métodos de seleção escolhidos, a organização de módulos e classes, a estratégia de busca de hiperparâmetros, as visualizações e a análise crítica.

- **Não é permitido:** copiar, traduzir ou reordenar *notebooks* públicos ou o trabalho de outra equipe; usar bibliotecas de AutoML (PyCaret, auto-sklearn, TPOT e similares); entregar código gerado integralmente por ferramentas de IA sem compreensão. O uso dessas ferramentas como apoio deve ser declarado no relatório, indicando o que foi gerado e como foi revisado.

Restrição de modelos - unidades 5 a 8

Naive Bayes, redes neurais artificiais, SVM e métodos *ensemble* (*Random Forest*, *Gradient Boosting*, *XGBoost*, *CatBoost*) **pertencem ao PP02 e não podem** ser usados como modelos deste projeto, nem mesmo como comparação. Se a equipe quiser mencionar tais técnicas, o lugar é a seção de trabalhos futuros, sem resultados.

3 A Base de Dados

- **Fonte:** UCI Machine Learning Repository, *dataset 697 – Predict Students’ Dropout and Academic Success* (<https://archive.ics.uci.edu/dataset/697>), DOI 10.24432/C5MC89.
- **Origem:** Instituto Politécnico de Portalegre (Portugal), cursos de graduação diversos, registros de 2008 a 2019.
- **Dimensões:** 4.424 instâncias e 36 atributos preditivos, sem valores ausentes declarados.
- **Alvo:** variável **Target**, com três categorias (Tabela 1).

Classe	Instâncias	Proporção	Significado
<i>Graduate</i>	2.209	≈ 49,9%	concluiu o curso
<i>Dropout</i>	1.421	≈ 32,1%	evadiu
<i>Enrolled</i>	794	≈ 17,9%	seguia matriculado, sem desfecho
Total	4.424	100%	

Tabela 1: Distribuição das classes. Os valores devem ser confirmados pela equipe na análise exploratória e reportados no relatório.

Grupo	Exemplos de atributos	Qtd.
Demográficos	estado civil, gênero, idade no ingresso, nacionalidade, deslocado	6
Socioeconômicos (família)	escolaridade e ocupação de pai e mãe	4
Socioeconômicos (aluno)	bolsista, devedor, mensalidade em dia, necessidades especiais	4
Ingresso	modo e ordem de candidatura, curso, turno, qualificação prévia e sua nota, nota de admissão	7
Desempenho – 1º semestre	unidades creditadas, inscritas, avaliações, aprovadas, nota média, sem avaliação	6
Desempenho – 2º semestre	(as mesmas seis do 1º semestre)	6
Macroeconômicos	taxa de desemprego, inflação, PIB	3
Total		36

Tabela 2: Agrupamento funcional dos atributos preditivos.

Atenção à codificação. Vários atributos categóricos (*Course*, *Application mode*, *Mother's occupation*, ...) vêm codificados como **inteiros arbitrários**. Tratá-los como numéricos faz o modelo assumir uma ordem que não existe e faz o k -NN calcular distâncias sem sentido entre cursos. Identificar quais colunas são realmente ordinais, quais são nominais e quais são contínuas é a **primeira tarefa não trivial** do projeto, e deve constar de uma tabela no relatório.

3.1 Os três cenários de predição

A base contém desempenho do 1º e do 2º semestres. Usar tudo produz a melhor acurácia e o pior sistema possível: quando o segundo semestre termina, boa parte da evasão já ocorreu, e a predição perde utilidade prática. Este é um caso clássico de vazamento temporal (*data leakage*).

A equipe deve, portanto, treinar e avaliar os modelos em **três recortes distintos** de atributos:

Cenário	Momento da predição	Grupos de atributos incluídos	Nº atrib.
A	No ato da matrícula (t_0)	demográficos, socioeconômicos, ingresso, macroeconômicos	24
B	Fim do 1º semestre	cenário A + desempenho do 1º sem.	30
C	Fim do 1º ano	cenário B + desempenho do 2º sem.	36

Tabela 3: Cenários obrigatórios. Todos os experimentos devem ser reportados nos três.

Espera-se que o desempenho cresça de A para C. A discussão exigida no relatório é: *quanto* se ganha em cada etapa, *quanto* se perde em capacidade de intervenção, e **qual cenário a equipe recomendaria à coordenação** com justificativa que não seja apenas “o de maior acurácia”.

4 Requisitos Técnicos

4.1 Unidade 1 - Modelagem do processo de ML

1. Caracterizar formalmente a tarefa: tipo de aprendizado, natureza do alvo, espaço de entrada $\mathcal{X}$ e de saída $\mathcal{Y}$, e a hipótese de que treino e teste vêm da mesma distribuição;
2. Descrever as etapas do *pipeline* adotado (aquisição $\rightarrow$ pré-processamento $\rightarrow$ representação $\rightarrow$ indução $\rightarrow$ avaliação $\rightarrow$ implantação), com um diagrama próprio;
3. Justificar a escolha entre manter as três classes ou reduzir o problema a binário. A tarefa principal é de três classes; a versão binária (*Dropout* $\times$ demais) é obrigatória apenas como análise secundária, e a equipe deve comentar por que a classe *Enrolled* é ambígua e o que se perde ao descartá-la;
4. Definir explicitamente a métrica de decisão do projeto, ou seja, o número único que a equipe usará para escolher o modelo final, e justificar a escolha diante do desbalanceamento.

4.2 Unidade 2 - Extração e Seleção de Características

a) Preparação e análise:

1. Análise exploratória: distribuição do alvo, estatísticas descritivas, detecção de *outliers*, correlação entre atributos numéricos e associação entre categóricos e o alvo;

2. Tabela de tipos (nominal / ordinal / contínuo) para os 36 atributos, com a codificação adotada para cada tipo (*one-hot*, ordinal, ou agrupamento de categorias raras);
3. Normalização: implementar min-max e padronização (*z-score*) e comparar.

b) Extração de características por PCA (obrigatório):

A Unidade 2 trata de *extração* e de *seleção*, que não são sinônimos: a seleção mantém um subconjunto dos atributos originais; a extração cria atributos novos como combinações dos originais.

1. Implementar a PCA a partir da matriz de covariância: centralização, decomposição em autovalores e autovetores, ordenação decrescente por autovalor e projeção $\mathbf{Z} = (\mathbf{X} - \bar{\mathbf{x}}) \mathbf{W}_d$. O uso de `numpy.linalg.eigh` ou `svd` é permitido, pois são primitivas de álgebra linear, não de ML. Validar contra o PCA do *scikit-learn*;
2. Padronizar antes de projetar. Sem padronização, a PCA é dominada pelos atributos de maior variância numérica $\rightarrow$ idade e notas esmagam os indicadores binários e as componentes perdem significado. Reportar o resultado **com** e **sem** padronização e explicar a diferença;
3. Apresentar o gráfico de variância explicada (*VE*) (individual e acumulada) e escolher o número de componentes d por um critério declarado: 90% ou 95% de variância acumulada ou cotovelo do *scree plot*;
4. Apresentar a projeção 2D nas duas primeiras componentes, colorida por classe, comentando a separabilidade observada em especial a sobreposição da classe *Enrolled*;
5. Discutir o tratamento das variáveis categóricas: aplicar PCA sobre colunas *one-hot* binárias é executável e conceitualmente frágil, porque variância de variável binária não tem a mesma interpretação. Justificar a decisão adotada – projetar apenas o bloco numérico e concatenar as categóricas, ou projetar tudo e assumir a limitação. MCA e FAMD podem ser citadas, sem implementação.

4.3 Unidade 3 – Árvores de Decisão

1. Implementar do zero entropia, ganho de informação e índice Gini, conferindo os valores contra um cálculo feito à mão para uma única divisão, apresentado no relatório;
2. Implementar do zero uma árvore ID3/CART simplificada, com divisão binária por limiar para atributos contínuos e parada por profundidade máxima e número mínimo de amostras na folha. Comparar o desempenho com o do `DecisionTreeClassifier`: resultados próximos bastam, não precisa chegar na coincidência exata das predições. A árvore usada nos experimentos da Seção 6 pode ser a do *scikit-learn*;
3. Mostrar o efeito da profundidade: um gráfico do desempenho em treino e em validação para `max_depth` crescente, indicando onde começa o sobreajuste.

4.4 Unidade 4 - Aprendizagem Baseada em Instâncias

1. Implementar o k -NN do zero (obrigatório, sem `KNeighborsClassifier` no núcleo), com voto majoritário e voto **ponderado pelo inverso da distância**, e validar contra o *scikit-learn*;

2. Implementar e comparar ao menos duas medidas de distância (euclidiana e Manhattan). Realizar uma pesquisa sobre as distâncias Chebyshev, Minkowski com p variável, ou cosseno), discutindo qual faz sentido para dados mistos;
3. Tratar explicitamente o problema de atributos heterogêneos: distância euclidiana sobre colunas categóricas codificadas como inteiros é matematicamente válida e semanticamente vazia. Adotar *one-hot*;
4. Varrer $k \in \{1, 3, 5, \dots, 51\}$ e apresentar a curva de desempenho em função de k , discutindo os extremos: $k = 1$ (variância alta) e k grande (viés alto, convergindo à classe majoritária);
5. Demonstrar experimentalmente duas propriedades vistas em aula:
 - (a) a normalização é decisiva para o k -NN e **irrelevante** para a árvore.
 - (b) Rodar ambos com e sem normalização e apresentar os quatro resultados numa mesma tabela;
 - (c) a maldição da dimensionalidade: medir o desempenho do k -NN com 5, 10, 20 e todos os atributos e comentar o comportamento observado. Repetir a medição com o mesmo número de **componentes principais** e comparar: reduzir dimensão *seleccionando* e reduzir *projetando* produzem o mesmo efeito sobre as distâncias?

4.5 Protocolo de Avaliação

1. **Seleção de modelo:** validação cruzada estratificada k -fold ($k = 10$), repetida 3 vezes, sobre o treino. Reportar sempre média $\pm$ desvio padrão;
2. **Vazamento:** normalização, codificação, imputação e seleção de atributos devem ser ajustadas dentro de cada *fold*, nunca antes da partição;
3. **Métricas:** acurácia, precisão, revocação e F1 por classe e em média macro, além da matriz de confusão. Na análise binária secundária, incluir AUC-ROC.

4.6 Análise crítica de viés (obrigatória)

A base contém atributos sensíveis: gênero, nacionalidade, estado civil, escolaridade e ocupação dos pais, condição de bolsista e de devedor. A equipe deve:

1. Reportar a **taxa de erro do modelo final por subgrupo** (no mínimo por gênero e por condição de bolsista), verificando se o sistema erra mais para uns do que para outros;
2. Discutir o risco de **profecia autorrealizável**: rotular um aluno como “provável evasão” pode alterar como ele é tratado pela instituição;
3. Posicionar-se sobre a inclusão de atributos socioeconômicos e familiares no modelo, avaliando ganho preditivo *versus* aceitabilidade ética de decidir com base neles.

5 Estrutura de Implementação

Bibliotecas de apoio (`pandas`, `numpy`, `matplotlib`, `seaborn`, `scipy`) são livres. O `scikit-learn` é permitido para partição, validação cruzada, métricas e como referência de validação. O núcleo dos dois classificadores, porém, é de implementação própria conforme as Seções 4.3 e 4.4. *Notebook* é aceito para exploração, mas o *pipeline* final deve rodar por *script*, com um único comando.

Listing 1: Assinaturas sugeridas (a implementacao e autoral)

```
1 # --- Dados e pre-processamento -----
2 def carregar_dados(caminho, cenario):
3     """Le o CSV e devolve X, y filtrados pelo cenario ('A', 'B' ou 'C')."""
4
5 def construir_atributos(df):
6     """Cria os atributos derivados. Tratar divisao por zero."""
7
8 def codificar(X, mapa_de_tipos):
9     """One-hot para nominais, ordinal para ordinais, passthrough contínuos."""
10
11 def normalizar(X_treino, X_teste, metodo):
12     """Ajusta o escalonador SO no treino e aplica nos dois conjuntos."""
13
14 # --- Unidade 2: selecao de caracteristicas -----
15 def pontuar_atributos(X, y, criterio):
16     """Devolve o score de cada atributo. criterio: 'mi', 'anova', 'chi2'."""
17
18 def selecionar_topk(scores, k):
19     """Devolve os indices dos k atributos mantidos."""
20
21 def pca_ajustar(X_treino, n_componentes):
22     """Centraliza, calcula a covariancia, decompoe e ordena por autovalor.
23     Devolve (media, W, variancia_explicada). Ajustar SO no treino."""
24
25 def pca_projetar(X, media, W):
26     """Projeta no subespaco de componentes principais: Z = (X - media) @ W."""
27
28 def cargas_componentes(W, nomes_atributos, n=3):
29     """Loadings das n primeiras componentes, para interpretacao no relatorio."""
30
31 # --- Unidade 3: arvore de decisao -----
32 def entropia(y):
33     """Entropia de Shannon do vetor de rotulos."""
34
35 def ganho_informacao(coluna, y, limiar):
36     """Ganho da divisao binaria coluna <= limiar."""
37
38 def induzir_arvore(X, y, criterio, prof_max, min_folha):
39     """ID3/CART simplificado. Devolve a raiz da arvore."""
40
41 def extrair_regras(no, nomes_atributos, prefixo=None):
42     """Percorre a arvore e devolve regras SE-ENTAO com cobertura e confianca."""
43
44 # --- Unidade 4: k-NN -----
45 def distancia(a, b, metrica, p=2):
46     """euclidiana | manhattan | chebyshev | minkowski | cosseno | heom."""
47
48 def classificar_knn(X_treino, y_treino, x, k, metrica, ponderado):
49     """Voto majoritario ou ponderado pelo inverso da distancia."""
50
51 # --- Avaliacao -----
52 def validacao_cruzada(modelo, X, y, n_folds, n_repeticoes, semente):
53     """Estratificada e repetida. Devolve o vetor de scores por fold."""
54
55 def matriz_confusao(y_real, y_pred, rotulos):
56     """Matriz KxK com os rotulos na ordem informada."""
57
58 def relatorio_metricas(y_real, y_pred):
59     """Acuracia, acuracia balanceada, P/R/F1 por classe e macro."""
60
```

```
61 # --- Execução ---  
62 def executar_experimento(cenario, conjunto_atributos, modelo, config):  
63     """Roda um ponto da grade experimental e persiste o resultado."""
```

6 Experimentos e Resultados (Obrigatórios)

Não basta o código rodar: a equipe deve medir e comparar. Use semente aleatória fixa e registre-a (Dica: 42). Todos os experimentos abaixo devem ser reportados para os **três cenários** da Tabela 3.

1. **E1 - Efeito da normalização:** tabela 2×2 (árvore / k -NN $\times$ com / sem normalização) e explicação do porquê do resultado;
2. **E2 - Análise de componentes principais:** gráfico de variância explicada acumulada, critério de escolha de d , projeção 2D colorida por classe, cargas das três primeiras componentes e **curva de desempenho em função do número de componentes** ($d = 2, 5, 10, 20, \dots$) para os **dois** classificadores no mesmo gráfico. Reportar também o efeito de omitir a padronização prévia;
3. **E3 - Medidas de distância:** comparação das métricas implementadas, com o k ótimo de cada uma;
4. **E4 - Custo computacional:** tempo de **treino** e tempo de **predição** de 1.000 instâncias, para os dois modelos. Espera-se o contraste entre aprendizado *ansioso* e *preguiçoso* e o efeito da redução de dimensionalidade sobre o custo de predição do k -NN. A equipe deve nomear e explicar ambos;
5. **E5 - Modelo final no teste:** escolhido o modelo pela validação cruzada, avaliá-lo **uma vez** no conjunto de teste. Apresentar matriz de confusão, métricas por classe e análise dos erros $\rightarrow$ em particular, o que acontece com a classe *Enrolled*;
6. **E6 - Interpretabilidade e viés:** Se o melhor modelo for o projetado por PCA, discutir explicitamente o custo dessa escolha para a coordenação;
7. **E7 - Curva de aprendizado (opcional, bônus):** desempenho em função do tamanho do treino, com a discussão sobre valer ou não a pena coletar mais dados.

7 Entregáveis

7.1 Código-fonte

Arquivo .zip (ou repositório Git público) contendo o código, um README com instruções de execução, a lista de dependências (`requirements.txt`), as sementes usadas e os artefatos de resultado (tabelas em .csv e figuras). O *pipeline* completo deve rodar sem erros a partir de um **único comando**. Não versionar a base de dados: o README deve indicar como obtê-la.

7.2 Relatório técnico (PDF)

O relatório deve ser um documento didático e de preferência escrito em L^AT_EX. Ademais, alguém que não viu o código precisa entender como o modelo decide e por que ele foi escolhido. Estrutura mínima:

1. **Identificação:** equipe, divisão de tarefas por integrante, declaração de uso de ferramentas de IA;

2. **Passo 0 - Fundamentação teórica:** breve fundamentação das técnicas utilizadas (entropia e ganho de informação, indução de árvores, poda, k -NN, medidas de distância, protocolos de validação);
3. **Passo 1 - Modelagem do problema:** tarefa, *pipeline*, escolha da métrica de decisão, custo dos erros, os três cenários;
4. **Passo 2 - Dados e características:** análise exploratória, tabela de tipos, atributos derivados, normalização, *rankings* de seleção e extração por PCA (variância explicada, projeção 2D e interpretação das cargas);
5. **Passo 3 - Árvore de decisão:** implementação, validação contra o *scikit-learn* e poda;
6. **Passo 4 - k -NN:** implementação, métricas de distância, curva de k , efeito da normalização e da dimensionalidade;
7. **Resultados:** tabelas e gráficos da Seção 6, com análise, não apenas números;
8. **Discussão:** qual modelo e qual cenário a equipe recomenda à coordenação, e por quê; análise de viés;
9. **Limitações e trabalhos futuros;**
10. **Referências.**

Extensão sugerida: 12 a 18 páginas.

O erro mais comum nesta entrega é o relatório-catálogo: dezenas de gráficos sem uma linha de interpretação. Toda tabela e toda figura devem vir acompanhadas da resposta a “o que isto mostra e o que muda por causa disso”. Um resultado **negativo** bem analisado por exemplo: “a seleção de atributos não melhorou o desempenho, e aqui está a hipótese do porquê” vale mais do que um número alto sem explicação.

7.3 Apresentação com os resultados

Slides (PDF) mais demonstração ao vivo do *pipeline*, de 10 a 15 minutos, com roteiro sugerido:

Slide	Conteúdo	Tempo
1	Equipe, problema e métrica de decisão escolhida	1 min
2	Dados: alvo, tipos de atributo e os três cenários	2 min
3	Características: derivadas, seleção e PCA (projeção 2D)	2 min
4	Árvore: implementação própria e regras extraídas	2 min
5	k -NN: curva de k , distância e efeito da normalização	2 min
6–7	Resultados: grade principal, teste final e custo	3 min
8	Recomendação à coordenação, viés e limitações	2 min

Tabela 4: Roteiro sugerido para a apresentação.

Leituras Recomendadas

- [1] HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning*. 2. ed. Springer, 2008. – Cap. 9 (árvores), 13 (protótipos e k -NN) e 7 (avaliação de modelos).
- [2] BISHOP, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006. – Cap. 1 e 14.4.
- [3] THEODORIDIS, S.; KOUTROUMBAS, K. *Pattern Recognition*. 4. ed. Academic Press, 2008. – Cap. 5 (seleção de características) e 6 (geração de características: transformada KL / PCA).
- [4] DUDA, R. O.; HART, P. E.; STORK, D. G. *Pattern Classification*. 2. ed. Wiley, 2000. – Cap. 3.8 (PCA), 4 (vizinho mais próximo) e 8 (árvores).
- [5] MARTINS, M. V.; TOLLEDO, D.; MACHADO, J.; BAPTISTA, L. M. T.; REALINHO, V. Early prediction of student's performance in higher education: a case study. *Trends and Applications in Information Systems and Technologies*, Springer, 2021.
- [6] REALINHO, V.; MACHADO, J.; BAPTISTA, L.; MARTINS, M. V. Predicting student dropout and academic success. *Data*, v. 7, n. 11, p. 146, 2022.

Bom trabalho e boa sorte!